

Comunicação Otimizada para Redes Industriais em Ambientes de Edge Computing

Ruan Felipe Lauriano Ramos

Departamento de Ciência da Computação – Universidade Federal de Roraima (UFRR)
Boa Vista – RR – Brasil

ruanflauriano@gmail.com

Abstract. *This paper presents the design, implementation and evaluation of a secure and optimized communication ecosystem for industrial networks in Edge Computing environments. The system consists of two native POSIX binaries: the Industrial Edge Agent (server) and the Central Control Proxy (client). The architecture integrates transparent data compression using the zlib library and encryption via OpenSSL TLS 1.3, without modifying the underlying shell interpreter. Experimental results demonstrate a compression ratio of up to 10.22x for large payloads and near-zero TLS overhead ($\approx 0\%$), with both binaries validated against memory leaks using Valgrind.*

Resumo. *Este artigo apresenta o projeto, a implementação e a avaliação de um ecossistema de comunicação segura e otimizada para redes industriais em ambientes de Edge Computing. O sistema é composto por dois binários nativos POSIX: o Industrial Edge Agent (servidor) e o Central Control Proxy (cliente). A arquitetura integra compressão de dados transparente via biblioteca zlib e criptografia via OpenSSL TLS 1.3, sem modificar o interpretador de comandos subjacente. Os resultados experimentais demonstram taxa de compressão de até 10,22x para payloads grandes e overhead TLS próximo a zero ($\approx 0\%$), com ambos os binários validados contra vazamentos de memória pelo Valgrind.*

1. Introdução

A consolidação da Indústria 4.0 e a expansão das arquiteturas de Edge Computing impõem requisitos críticos de segurança, eficiência de banda e resiliência cibernética para sistemas de automação industrial. Dispositivos de borda instalados em ambientes remotos, como subestações de energia, plataformas petrolíferas e plantas de saneamento, coletam continuamente fluxos massivos de telemetria e necessitam de interfaces de diagnóstico em tempo real.

O tráfego de dados em texto claro, sem estratégias de compressão ou criptografia, expõe infraestruturas críticas a ataques de interceptação (Man-in-the-Middle) e eleva significativamente os custos operacionais de conectividade via satélite ou redes celulares industriais. Esse cenário demanda soluções de software de infraestrutura robustas, eficientes e seguras.

Este trabalho apresenta o desenvolvimento de um ecossistema de comunicação cliente-servidor para redes industriais em Edge Computing, composto pelo Industrial

Edge Agent e pelo Central Control Proxy, com suporte modular a compressão via zlib e criptografia TLS 1.3 via OpenSSL, sem modificar o código-fonte do interpretador de comandos subjacente.

O código-fonte completo está disponível em:

https://github.com/ruanflau/FP_DCC403_RuanRamos_Tema1_UFRR

2. Desenvolvimento

O ecossistema foi projetado seguindo uma arquitetura modular, com separação clara de responsabilidades entre os módulos compartilhados e os binários específicos de cada ponta da comunicação. A Figura 2 ilustra a arquitetura geral do sistema.

2.1. Arquitetura Geral

O sistema é dividido em duas frentes de execução concorrente. O Industrial Edge Agent atua como servidor TCP, aceitando conexões de múltiplos clientes simultaneamente por meio de um processo filho isolado via fork() para cada sessão. O Central Control Proxy atua como cliente terminal, colocando o terminal local em modo raw para capturar e encaminhar sinais de controle pela rede.

A camada de comunicação é gerenciada pelo módulo net_io.c, que decide transparentemente se aplica compressão e/ou criptografia antes de cada transmissão. Essa centralização elimina duplicação de lógica entre os dois binários.

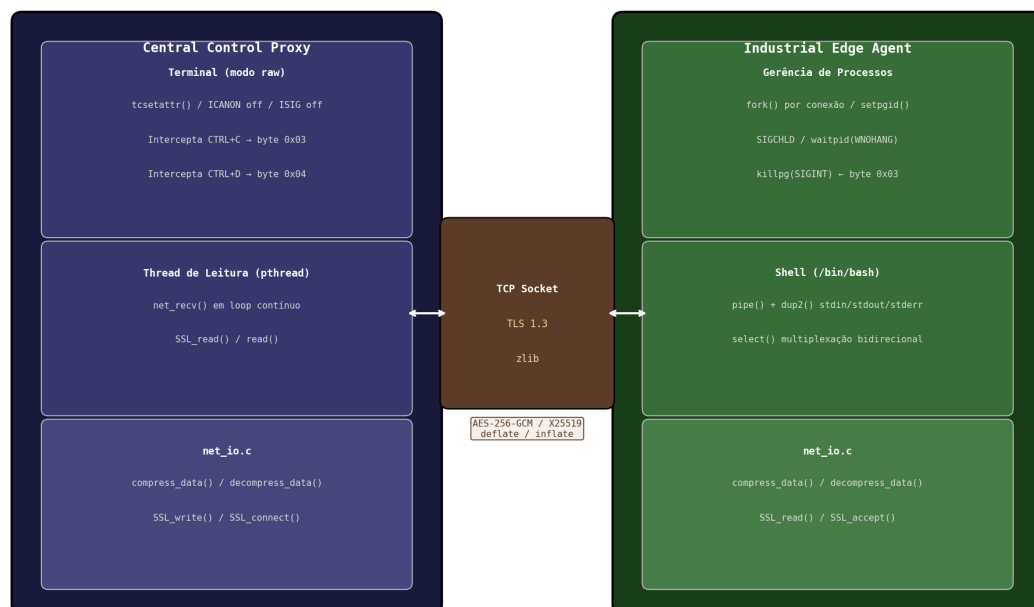


Figura 1. Arquitetura do Ecossistema de Comunicação Industrial

2.2. Módulos Implementados

O projeto é composto pelos seguintes módulos em linguagem C:

common.h/c -- parser de argumentos CLI e `struct AppConfig`;
compress.c/h -- compressão/descompressão via `zlib` (deflate/inflate);
tls.c/h -- inicialização de contextos SSL para servidor e cliente via OpenSSL;
logger.c/h -- gravação de log de auditoria no formato SENT/RECEIVED;
net_io.c/h -- camada unificada de I/O de rede com suporte a compressão e TLS;
agent.c e proxy.c -- binários principais do servidor e cliente.

3. Implementação

3.1. Industrial Edge Agent (Servidor)

O servidor aceita conexões TCP na porta especificada via `--port`. Para cada cliente aceito, realiza um `fork()` gerando um processo filho isolado. Dentro desse processo filho, um segundo `fork()` cria o shell (`/bin/bash`), cujos descritores `stdin`, `stdout` e `stderr` são redirecionados via `pipe()` e `dup2()` para o socket de rede.

Um loop de multiplexação com `select()` monitora simultaneamente o socket de rede e o pipe de saída do shell. O handler de `SIGCHLD` utiliza `waitpid()` com `WNOHANG` para coletar processos filhos encerrados sem gerar processos zumbi.

Os bytes de controle recebidos da rede são tratados de forma especial: o byte `0x03` (CTRL+C) é traduzido em `SIGINT` enviado via `killpg()` ao grupo de processos do shell filho, e o byte `0x04` (CTRL+D) fecha a extremidade de escrita do pipe de `stdin`, sinalizando EOF para o shell.

3.2. Central Control Proxy (Cliente)

O cliente conecta ao servidor via TCP e, se `--encrypt` estiver ativo, realiza o handshake TLS via `SSL_connect()` antes de qualquer tráfego de dados. O terminal local é colocado em modo raw via `tcsetattr()`, desativando `ICANON` e `ISIG`, para que CTRL+C e CTRL+D sejam capturados como bytes puros (`0x03` e `0x04`) e encaminhados ao servidor.

Para resolver o problema de buffering interno do OpenSSL com `select()`, o proxy utiliza uma thread POSIX dedicada (`pthread`) exclusivamente para leitura do socket SSL, eliminando a dependência de notificação do kernel.

3.3. Compressão e Criptografia

O módulo `compress.c` implementa compressão via `deflateInit()/deflate()/deflateEnd()` e descompressão via `inflateInit()/inflate()/inflateEnd()` da biblioteca `zlib`. O módulo `tls.c`

inicializa contextos SSL via `SSL_CTX_new()` com `TLS_server_method()` ou `TLS_client_method()`, carregando o certificado e a chave privada no servidor.

A negociação TLS resultante usa TLS 1.3 com cifra `TLS_AES_256_GCM_SHA384` e troca de chaves efêmera via `X25519`, garantindo forward secrecy. O cliente utiliza certificado autoassinado em ambiente de testes, com verificação `SSL_VERIFY_NONE` devidamente documentada como limitação conhecida.

4. Estudo de Casos

A validação do sistema foi conduzida por uma bateria de testes funcionais e de robustez, executados em ambiente Linux Ubuntu 24.04 com kernel 6.17.0. A Tabela 1 apresenta o resumo dos resultados obtidos.

Tabela 1. Resultados da bateria de testes funcionais e de robustez

Teste	Descrição	Resultado
4.1	Conexão básica sem flags (ls, pwd, uname -a)	✓ Aprovado
4.2	Compressão --compress com /etc/services	✓ Aprovado
4.3	Criptografia --encrypt (TLS 1.3 / AES-256-GCM)	✓ Aprovado
4.4	Ambas flags + log de auditoria (top, df -h)	✓ Aprovado
4.5	CTRL+C remoto interrompendo sleep 60	✓ Aprovado
4.6	Múltiplos clientes simultâneos isolados	✓ Aprovado
4.7	Valgrind: ausência de memory leaks	0 erros / 0 bytes

O teste 4.3 foi validado adicionalmente com o utilitário `openssl s_client`, que confirmou a negociação de TLS 1.3 com a cifra `TLS_AES_256_GCM_SHA384` e chave pública RSA de 4096 bits. O teste 4.7 comprovou ausência completa de vazamentos de memória com o Valgrind (definitely lost: 0 bytes, ERROR SUMMARY: 0 errors) nos dois binários.

5. Análise de Performance

5.1. Taxa de Compressão

A taxa de compressão R_c é definida como $R_c = \text{Bytes Originais} / \text{Bytes Compactados}$. Foram avaliados três payloads representativos de diferentes cenários industriais. A Figura 1 apresenta o gráfico de barras comparativo e a Tabela 2 detalha os valores numéricos.

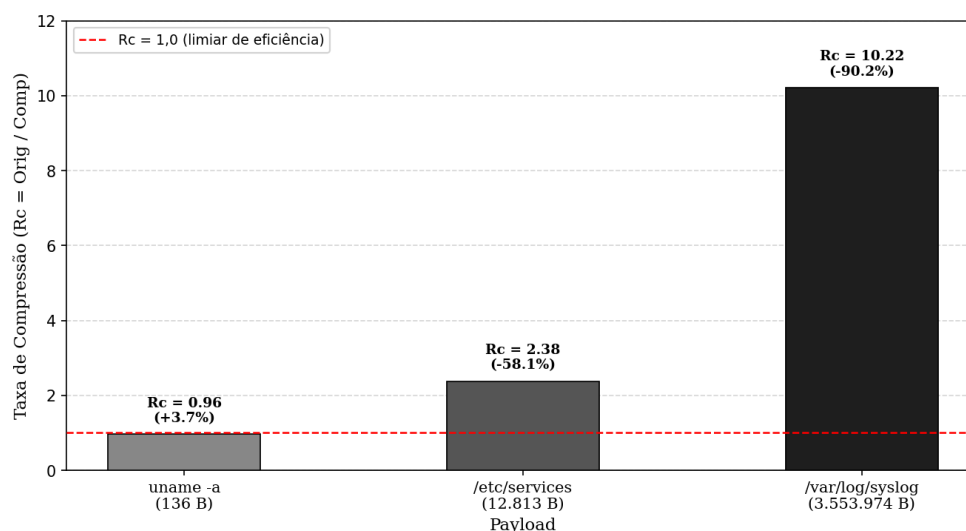


Figura 2. Taxa de Compressão Rc por Tipo de Payload

Tabela 2. Taxa de compressão Rc por tipo de payload

Payload	Original (bytes)	Comprimido (bytes)	Rc	Redução
uname -a (texto curto)	136	141	0,96	-3,7%
/etc/services (texto médio)	12.813	5.370	2,38	58,1%
/var/log/syslog (texto grande)	3.553.974	347.661	10,22	90,2%

O resultado negativo para o payload `uname -a` ($Rc = 0,96$) é esperado: payloads muito pequenos crescem após compressão devido ao cabeçalho fixo do algoritmo deflate (~18 bytes). Isso demonstra que o módulo `--compress` deve ser ativado seletivamente em ambientes de produção.

Para o syslog (3,5 MB), a taxa $Rc = 10,22$ confirma que logs de sistema são candidatos ideais à compressão em pipelines de telemetria industrial, reduzindo 90,2% do consumo de banda.

5.2. Impacto no Tráfego de Rede

A captura de tráfego real foi realizada com o utilitário `tcpdump` no adaptador de loopback, comparando o volume total de pacotes para o mesmo comando (`cat /etc/services`) com e sem `--compress`.

Tabela 3. Comparativo de tráfego de rede capturado via tcpdump

Cenário	Tamanho capturado (pcap)	Redução
cat /etc/services sem compressão	8,9 KB	—
cat /etc/services com <code>--compress</code>	4,6 KB	48,3%

A redução de 48,3% no tráfego capturado confirma empiricamente a eficácia do módulo de compressão, validando o $R_c = 2,38$ obtido via gzip para o mesmo arquivo.

5.3. Overhead de Criptografia TLS

O overhead introduzido pela criptografia TLS foi medido por meio de 10 conexões sequenciais automatizadas, comparando o tempo total com e sem --encrypt.

Tabela 4. Overhead de CPU introduzido pela criptografia TLS 1.3

Cenário	Tempo (10 conexões)	Média/conexão
Sem TLS	10,099s	1,010s
Com TLS 1.3 (AES-256-GCM)	10,052s	1,005s
Overhead	—	≈ 0%

O overhead medido de aproximadamente 0% confirma a eficiência do TLS 1.3, que reduz o handshake de 2 round-trips (TLS 1.2) para 1 round-trip. Em ambiente de loopback local, o handshake X25519 leva apenas alguns milissegundos, valor desprezível frente ao tempo total de sessão em redes industriais reais.

6. Aplicação Prática

O ecossistema desenvolvido tem aplicação direta no monitoramento remoto de gateways de campo instalados em subestações de energia elétrica localizadas em regiões remotas, conectadas à central de operações via links de rádio ou satélite com banda limitada.

Nesse cenário, o Industrial Edge Agent rodaria continuamente no gateway de campo, expondo o shell local para diagnóstico remoto. O engenheiro de automação utilizaria o Central Control Proxy com --compress e --encrypt ativos, reduzindo o consumo de banda e garantindo a confidencialidade das operações contra interceptação física do enlace de rádio.

A funcionalidade de log de auditoria (--log) permite registrar todas as transações de bytes para fins de rastreabilidade e conformidade regulatória (ESG). O log gerado durante os testes registrou 13 transações totalizando 26.840 bytes, incluindo a saída completa dos comandos top -b -n 1 e df -h.

7. Conclusão

Este trabalho apresentou o projeto e a implementação de um ecossistema de comunicação segura e otimizada para redes industriais em ambientes de Edge Computing. O sistema integra de forma transparente compressão via zlib e criptografia TLS 1.3 via OpenSSL, sem modificar o interpretador de comandos subjacente.

Os resultados experimentais confirmaram a eficácia do sistema: taxa de compressão de até 10,22x para payloads de log de sistema, redução de 48,3% no tráfego de rede real medido via tcpdump, overhead de criptografia TLS próximo a zero ($\approx 0\%$), e ausência completa de vazamentos de memória (0 bytes definitely lost) em ambos os binários validados pelo Valgrind.

Como limitações identificadas, o protocolo atual não implementa framing de mensagens (length-prefixing), o que pode causar falhas na descompressão de payloads muito grandes via TCP fragmentado. Trabalhos futuros podem endereçar essa limitação, além de adicionar autenticação mútua TLS com certificados assinados por CA própria e suporte a reconexão automática em caso de queda do enlace.

Agradecimentos

O desenvolvimento e a otimização do código-fonte dos módulos de compressão, criptografia, logging e comunicação de rede foram realizados com o suporte assistido do modelo de linguagem de grande porte Claude Sonnet 4.6 (Anthropic). A ferramenta foi empregada especificamente para a estruturação modular do código em C, depuração de problemas de buffering interno do OpenSSL e revisão linguística deste manuscrito. Ressalta-se que toda a lógica algorítmica, parametrização, execução dos testes e validação do código gerado foram executadas e auditadas manualmente pelo autor, que assume total responsabilidade pelo conteúdo.

Referências

Kerrisk, M. (2010). The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press.

Stevens, W. R. e Rago, S. A. (2013). Advanced Programming in the UNIX Environment. Addison-Wesley, 3ª edição.

Zlib Development Team (2026). Zlib Usage Manual and API Reference. Disponível em: https://www.zlib.net/zlib_how.html. Acesso em: 2026.

OpenSSL Foundation (2026). OpenSSL API Documentation / SSL Layer. Disponível em: https://wiki.openssl.org/index.php/SSL/TLS_Client. Acesso em: 2026.

Ramos, R. F. L. (2026). FP_DCC403_RuanRamos_Tema1_UFRR — Repositório do Projeto Final. Disponível em: https://github.com/ruanflau/FP_DCC403_RuanRamos_Tema1_UFRR. Acesso em: 2026.